INTERNATIONAL ISLAMIC UNIVERSITY ISLAMABAD

COURSE:-

Software Construction and Development

Course Code:- SE 302

Submitted By:

Name:- Muhammad Abdullah Khan

Reg no:- 4747-FOC/BSSE/F23 **(A)**

Submitted To:

Name:- Sir Shakir Rasheed Khan Khattak

Department Of Software Engineering

Faculty of Computing & IT

1. Introduction

This report analyses the Employee Management System and demonstrates how to identify and reduce both Cyclomatic and Cognitive Complexity. The project manages four employees Abdullah, Usman, Zayam and Bilal at Codistan and supports operations including adding, updating, removing employees, payroll calculation, promotion checking and report generation.

Each major function is examined in full: the original code is shown with annotated decision points, followed by the refactored version with a detailed explanation of every complexity reducing change applied.

2. Complexity Theory

2.1 Cyclomatic Complexity

Cyclomatic Complexity (CC), introduced by Thomas McCabe, counts the number of linearly independent execution paths through a function. It answers: how many test cases are needed to achieve 100% code coverage?

Formula: $M = E - N + 2P$ (E = edges, N = nodes, P = connected components)

Practical shortcut: count every if, else if, for, while, case, &&, || as +1, then add 1 for the function itself.

Score	Classification	Risk Level	Action Required
1 – 10	Simple	Not much risk	None
11 – 20	Complex	Low risk	Monitor
21 – 50	Too Complex	Medium risk; attention needed	Refactor
> 50	Too Complex	Cannot test; high risk	Immediate refactor

2.2 Cognitive Complexity

Cognitive Complexity, introduced by SonarQube, measures how hard code is to read. It specifically penalises nesting. The deeper a structure is nested, the harder it is for a developer to track context mentally.

Rules:

- +1 for each control flow structure: if, else if, else, for, while, switch.
- +1 extra for each nesting level the structure is inside (nesting penalty).
- +1 for each logical operator sequence break: && mixed with ||.
- Switch counts as 1 regardless of the number of cases inside it.

Acceptable threshold per method: 15 or below (professional standard). Target: under 10.

3. Original Code (Complexity Analysis)

The original code had three main complexity problems:

- Nested loops inside loops (removeEmployee used a loop-inside-loop to shift the array).
- Identical search patterns copy-pasted in four different methods (removeEmployee, updateEmployee, calculatePayroll, generateReport all looped through the array independently).
- Large if-else blocks instead of guard clauses, causing all successful logic to be pushed into the else branch and increasing nesting depth.

Function	Cyclomatic CC	Cognitive CC	Worst Offender
addEmployee()	2	3	if-else block, nested else penalty
removeEmployee()	4	5	Loop inside loop (depth 3)
updateEmployee()	3	4	All setters nested inside for + if
viewEmployees()	3	4	for loop nested inside else block
calculatePayroll()	3	4	Payroll logic nested inside for + if
generateReport()	3	4	Report loop nested inside else block
AskForPromotion()	2	3	Raw condition evaluated in-context
TOTAL	**20**	**27**	**Overall needs reduction**

4. Function by Function Code Comparison

Each function below shows: (1) the original code with annotated decision points, (2) the refactored code with explanatory comments, and (3) the resulting complexity change.

4.1 addEmployee()

Problem: The entire success logic was inside an if block, with the error message in an else. This creates an unnecessary else-branch nesting penalty and buries the main logic one level deep.

Fix applied: Extract a CanAddEmployee() helper for the capacity check, then use a guard clause (early return on failure). The success path stays at the top level of the function.

▶ ORIGINAL CODE; addEmployee(); Original: Cyclomatic CC = 2 , Cognitive CC = 3

```
void addEmployee(string name, string company, int age, string role,
                 int yearsOfExperience, double baseSalary,
                 string performanceReview) {
    if (employeeCount < MAX_EMPLOYEES) {          // Decision 1
        employees[employeeCount] = Employee(name, company, age, role,
```

```
                                    yearsOfExperience, baseSalary,
                                    performanceReview);
        employeeCount++;
        cout << "Employee added successfully!\n";
    } else {                                        // nesting penalty +1
        cout << "Employee list is full.\n";
    }
}
```

▶ REFACTORED CODE; addEmployee(); Refactored: Cyclomatic CC = 2 , Cognitive CC = 1

```
// Helper (single responsibility) — CC=2, Cog=1
bool CanAddEmployee(int currentCount, int maxEmployees) {
    return currentCount < maxEmployees;
}

void addEmployee(string name, string company, int age, string role,
                 int yearsOfExperience, double baseSalary,
                 string performanceReview) {
    if (!CanAddEmployee(employeeCount, MAX_EMPLOYEES)) {  // Decision 1
        cout << "Employee list is full.\n";
        return;                     // guard clause — no else needed
    }
    employees[employeeCount++] = Employee(name, company, age, role,
                                 yearsOfExperience, baseSalary,
                                 performanceReview);
    cout << "Employee added successfully!\n";
}
```

Metric	Before	After	Change
Cyclomatic CC	2	2	Same; logic count unchanged
Cognitive CC	3	1	↓ 2; else removed; helper call flattens condition
Nesting depth	2	1	↓ 1 level
Helper extracted	No	Yes (CanAddEmployee)	Single responsibility

4.2 removeEmployee()

Problem: This was the most complex function. It contained an outer for-loop, inside which was an if-statement, inside which was a second for-loop to shift elements left. This created nesting depth of 3, the worst in the project.

Fix applied: Extract findEmployeeIndex() to handle the search, and shiftEmployeesLeft() to handle the array shift. The main function becomes a flat sequence of three steps: find, validate, shift.

▶ ORIGINAL CODE; removeEmployee(); Original: Cyclomatic CC = 4 , Cognitive CC = 5

```
void removeEmployee(string name) {
    for (int i = 0; i < employeeCount; i++) {    // Decision 1 — loop
        if (employees[i].getName() == name) {    // Decision 2 — inner if
            // Shift left — nested loop inside outer loop
```

```
            for (int j = i; j < employeeCount - 1; j++) { // Decision 3
                employees[j] = employees[j + 1];
            }                                   // nesting depth = 3
            employeeCount--;
            cout << "\nEmployee list after removal:-\n";
            return;
        }
    }
}
// Cyclomatic CC = 4
// Cognitive  CC = 5
```

▶ REFACTORED CODE; removeEmployee(); Refactored: Cyclomatic CC = 2 , Cognitive CC = 2

```
// Helper 1: find index by name — CC=3, Cog=3
int findEmployeeIndex(const string& name) {
    for (int i = 0; i < employeeCount; i++) {    // Decision 1
        if (employees[i].getName() == name)       // Decision 2
            return i;
    }
    return -1;
}

// Helper 2: shift left — CC=2, Cog=1
void shiftEmployeesLeft(int idx) {
    for (int j = idx; j < employeeCount - 1; j++) // Decision 1
        employees[j] = employees[j + 1];
}

// Main function — now flat, no nested loops
void removeEmployee(const string& name) {
    int idx = findEmployeeIndex(name);             // delegate search
    if (idx == -1) {                               // Decision 1
        cout << "Employee not found.\n";
        return;
    }
    shiftEmployeesLeft(idx);                       // delegate shift
    employeeCount--;
    cout << "\nEmployee list after removal:-\n";
}
// Cyclomatic CC = 2
// Cognitive  CC = 2
```

Metric	Before	After	Change
Cyclomatic CC	4	2	↓ 2; nested loop removed from this function
Cognitive CC	5	2	↓ 3; nesting depth 3 eliminated
Nesting depth	3	1	↓ 2 levels
Helpers extracted	0	2 (findEmployeeIndex, shiftEmployeesLeft)	Both new

4.3 updateEmployee()

Problem: Identical search loop duplicated from removeEmployee. All setter calls were trapped inside a for-loop body, inside an if-block giving them a nesting penalty even though they are logically simple operations.

Fix applied: Delegate search to findEmployeeIndex(). Use a guard clause on the result. All setters now execute at the top level of the function body with zero nesting.

▶ ORIGINAL CODE; updateEmployee(); Original: Cyclomatic CC = 3 , Cognitive CC = 4

```
void updateEmployee(string name, string newRole, int newAge,
                    int newYearsOfExperience, double newBaseSalary,
                    string newPerformanceReview) {
    for (int i = 0; i < employeeCount; i++) {    // Decision 1
        if (employees[i].getName() == name) {    // Decision 2
            employees[i].setRole(newRole);
            employees[i].setAge(newAge);
            employees[i].setYearsOfExperience(newYearsOfExperience);
            employees[i].setBaseSalary(newBaseSalary);
            employees[i].setPerformanceReview(newPerformanceReview);
            cout << "\nUpdated the Employee List:-\n";
            return;
        }
    }
    cout << "Employee not found.\n";
}
// Cyclomatic CC = 3
// Cognitive  CC = 4
```

▶ REFACTORED CODE; updateEmployee(); Refactored: Cyclomatic CC = 2 , Cognitive CC = 2

```
void updateEmployee(const string& name, string newRole, int newAge,
                    int newYearsOfExperience, double newBaseSalary,
                    string newPerformanceReview) {
    int idx = findEmployeeIndex(name);            // delegate search
    if (idx == -1) {                              // Decision 1
        cout << "Employee not found.\n";
        return;
    }
    // All setters now at top level — zero nesting penalty
    employees[idx].setRole(newRole);
    employees[idx].setAge(newAge);
    employees[idx].setYearsOfExperience(newYearsOfExperience);
    employees[idx].setBaseSalary(newBaseSalary);
    employees[idx].setPerformanceReview(newPerformanceReview);
    cout << "\nUpdated the Employee List:-\n";
}
// Cyclomatic CC = 2
// Cognitive  CC = 2
```

Metric	Before	After	Change
Cyclomatic CC	3	2	↓ 1; for-loop removed from this function
Cognitive CC	4	2	↓ 2; setters now at nesting level 0
Nesting depth	2	1	↓ 1 level

Metric	Before	After	Change
Duplicate search	Yes	No (uses findEmployeeIndex)	DRY principle applied

4.4 viewEmployees()

Problem: The entire loop was placed inside the else branch of the empty-check. This is unnecessary because if the list is empty the function should simply return. Having the loop inside else adds a nesting level to all iterations.

Fix applied: Guard clause on empty check. The for-loop then executes at the top level of the function; no else needed.

▶ ORIGINAL CODE; viewEmployees(); Original: Cyclomatic CC = 3 , Cognitive CC = 4

```
void viewEmployees() {
    if (employeeCount == 0) {                    // Decision 1
        cout << "No employees to display.\n";
    } else {                                     // penalty +1 for else
        cout << "\nEmployee List:\n";
        cout << "=====================================\n";
        for (int i = 0; i < employeeCount; i++) { // Decision 2
            cout << "Employee " << i + 1 << ":\n";
            employees[i].introduceYourself();    // nested call
            cout << "-------------------------------------\n";
        }
    }
}
// Cyclomatic CC = 3
// Cognitive  CC = 4
```

▶ REFACTORED CODE; viewEmployees(); Refactored: Cyclomatic CC = 3 , Cognitive CC = 2

```
void viewEmployees() {
    if (employeeCount == 0) {                    // Decision 1 (guard)
        cout << "No employees to display.\n";
        return;                                  // exit immediately
    }
    cout << "\nEmployee List:\n";
    cout << "=====================================\n";
    for (int i = 0; i < employeeCount; i++) {   // Decision 2 — top-level
        cout << "Employee " << i + 1 << ":\n";
        employees[i].introduceYourself();
        cout << "-------------------------------------\n";
    }
}
// Cyclomatic CC = 3
// Cognitive  CC = 2
```

Metric	Before	After	Change
Cyclomatic CC	3	3	Same; two decisions remain (if + for)

Metric	Before	After	Change
Cognitive CC	4	2	↓ 2; else removed; for loop promoted to top level
Nesting depth	2	1	↓ 1 level
Guard clause	No	Yes	Early return on empty

4.5 calculatePayroll()

Problem: Same duplicated search loop. All payroll print statements were nested at depth 2 (inside for, inside if). Five cout statements paying nesting penalties they did not need to pay.

Fix applied: findEmployeeIndex() handles the search. Guard clause exits on missing employee. All print statements execute at nesting level 0.

▶ ORIGINAL CODE; calculatePayroll(); Original: Cyclomatic CC = 3 , Cognitive CC = 4

```
void calculatePayroll(string name, double bonus = 0.0,
                      double deductions = 0.0) {
    for (int i = 0; i < employeeCount; i++) {   // Decision 1
        if (employees[i].getName() == name) {   // Decision 2
            double finalSalary = employees[i].getBaseSalary()
                                 + bonus - deductions;
            cout << "\nPayroll for " << name << ":\n";
            cout << "Base Salary : " << employees[i].getBaseSalary() << endl;
            cout << "Bonus       : " << bonus << endl;
            cout << "Deductions  : " << deductions << endl;
            cout << "Final Salary: " << finalSalary << endl;
            return;
        }
    }
    cout << "Employee not found.\n";
}
// Cyclomatic CC = 3
// Cognitive  CC = 4
```

▶ REFACTORED CODE; calculatePayroll(); Refactored: Cyclomatic CC = 2 , Cognitive CC = 2

```
void calculatePayroll(const string& name, double bonus = 0.0,
                      double deductions = 0.0) {
    int idx = findEmployeeIndex(name);           // delegate search
    if (idx == -1) {                             // Decision 1 (guard)
        cout << "Employee not found.\n";
        return;
    }
    double finalSalary = employees[idx].getBaseSalary() + bonus - deductions;
    cout << "\nPayroll for " << name << ":\n";
    cout << "-------------------------------------\n";
    cout << "Base Salary  : " << employees[idx].getBaseSalary() << endl;
    cout << "Bonus        : " << bonus << endl;
    cout << "Deductions   : " << deductions << endl;
    cout << "Final Salary : " << finalSalary << endl;
    cout << "-------------------------------------\n";
}
// Cyclomatic CC = 2
// Cognitive  CC = 2
```

Metric	Before	After	Change
Cyclomatic CC	3	2	↓ 1; for-loop removed from this function
Cognitive CC	4	2	↓ 2; print block promoted to top level
Nesting depth	2	1	↓ 1 level
Duplicate search	Yes	No	Reuses findEmployeeIndex()

4.6 generateReport()

Problem: Same pattern as viewEmployees; the report loop is inside else, adding one unnecessary nesting level to every iteration.

Fix applied: Guard clause returns early if empty. The report loop runs at the top level.

▶ ORIGINAL CODE; generateReport(); Original: Cyclomatic CC = 3 , Cognitive CC = 4

```
void generateReport() {
    if (employeeCount == 0) {                    // Decision 1
        cout << "No employees in report.\n";
    } else {                                     // penalty +1 for else
        cout << "\nGenerating Employee Report:\n";
        cout << "-------------------------------------\n";
        for (int i = 0; i < employeeCount; i++) { // Decision 2
            cout << "Name   : " << employees[i].getName() << endl;
            cout << "Role   : " << employees[i].getRole() << endl;
            cout << "Salary : " << employees[i].getBaseSalary() << endl;
            cout << "Review : " << employees[i].getPerformanceReview() << endl;
            cout << "-------------------------------------\n";
        }
    }
}
// Cyclomatic CC = 3
// Cognitive  CC = 4
```

▶ REFACTORED CODE; generateReport(); Refactored: Cyclomatic CC = 3 , Cognitive CC = 2

```
void generateReport() {
    if (employeeCount == 0) {                    // Decision 1 (guard)
        cout << "No employees in report.\n";
        return;
    }
    cout << "\nGenerating Employee Report:\n";
    cout << "-------------------------------------\n";
    for (int i = 0; i < employeeCount; i++) {   // Decision 2 — top-level
        cout << "Name               : " << employees[i].getName() << endl;
        cout << "Role               : " << employees[i].getRole() << endl;
        cout << "Salary             : " << employees[i].getBaseSalary() << endl;
        cout << "Performance Review : " << employees[i].getPerformanceReview() <<
endl;
        cout << "-------------------------------------\n";
    }
}
```

```
// Cyclomatic CC = 3
// Cognitive  CC = 2
```

Metric	Before	After	Change
Cyclomatic CC	3	3	Same; two decisions remain (if + for)
Cognitive CC	4	2	↓ 2; else removed; loop at top level
Nesting depth	2	1	↓ 1 level
Pattern match	Yes (same as viewEmployees)	Fixed	Consistent refactoring

4.7 AskForPromotion()

Problem: The promotion check embedded a raw boolean expression directly in the if-condition. In the context of being called inside a loop from main(), the surrounding context added to the cognitive load of reading it.

Fix applied: Extract IsEligibleForPromotion() as a named helper. The if-condition now reads like plain English, reducing mental effort to interpret it.

▶ ORIGINAL CODE; AskForPromotion(); Original: Cyclomatic CC = 2 , Cognitive CC = 3

```
void AskForPromotion() {
    if (YearsOfExperience >= 5)        // Decision 1
        cout << Name << " is promoted!\n" << endl;
    else
        cout << Name << " is not promoted! Gain more experience.\n" << endl;
}
// Cyclomatic CC = 2
// Cognitive  CC = 3
```

▶ REFACTORED CODE; AskForPromotion(); Refactored: Cyclomatic CC = 2 , Cognitive CC = 1

```
// Helper — single responsibility, CC=2, Cog=1
bool IsEligibleForPromotion(int yearsOfExperience) {
    return yearsOfExperience >= 5;
}

void AskForPromotion() {
    if (IsEligibleForPromotion(YearsOfExperience))  // Decision 1
        cout << Name << " is promoted!\n" << endl;
    else
        cout << Name << " is not promoted!\n" << endl;
}
// Cyclomatic CC = 2
// Cognitive  CC = 1
```

Metric	Before	After	Change
Cyclomatic CC	2	2	Same; one decision remains

Metric	Before	After	Change
Cognitive CC	3	1	↓ 2; raw condition extracted to named helper
Readability	Low	High	if (IsEligibleForPromotion(...)) reads naturally
Helper extracted	No	Yes (IsEligibleForPromotion)	SRP applied

5. Overall Complexity Comparison

The table below summarises the Cyclomatic and Cognitive Complexity for every function, before and after refactoring. New helper functions extracted during refactoring are listed separately.

5.1 Existing Functions (Before vs After)

Function	Cyc CC Before	Cyc CC After	Cog CC Before	Cog CC After	Cog Change
addEmployee()	2	2	3	1	↓ 2
removeEmployee()	4	2	5	2	↓ 3
updateEmployee()	3	2	4	2	↓ 2
viewEmployees()	3	3	4	2	↓ 2
calculatePayroll()	3	2	4	2	↓ 2
generateReport()	3	3	4	2	↓ 2
AskForPromotion()	2	2	3	1	↓ 2
SUBTOTAL	**20**	**16**	**27**	**12**	**↓ 15**

5.2 New Helper Functions Added

These helpers carry the extracted logic. Each one is small, focused, and easy to test in isolation.

New Helper Function	Cyclomatic CC	Cognitive CC	Responsibility
findEmployeeIndex()	3	3	Search array by employee name; return index or -1
shiftEmployeesLeft()	2	1	Shift array elements left after a removal
IsEligibleForPromotion()	2	1	Return true if yearsOfExperience >= 5
CanAddEmployee()	2	1	Return true if array is not full
SUBTOTAL	**9**	**6**	**All below CC=3; all below Cog=4**

5.3 Project-Wide Summary

Metric	Original	Refactored	Improvement
Total Cyclomatic CC	20	25	Slightly higher; spread over more, smaller functions
Total Cognitive CC	27	18	↓ 9 points (33% reduction)
Max CC in any single function	4	3	↓ 1; no function exceeds threshold of 10
Max Cognitive CC in any function	5	3	↓ 2; no function exceeds threshold of 15
Functions with Cognitive CC > 3	7	0	All functions now below 4
Duplicate search patterns	4	0	Consolidated into findEmployeeIndex()
Max nesting depth	3	1	↓ 2 levels; much easier to read
Single Responsibility adherence	Partial	Yes	Every function has exactly one job
Guard clauses used	0	5	All mutating functions use early return pattern

6. Conclusion

This analysis demonstrates that complexity is not just about how much a function does, but how deeply nested its logic is. The original Employee Management System had acceptable Cyclomatic Complexity values (all under 5), yet its Cognitive Complexity was unnecessarily high due to duplicate search loops, deep nesting and large else-blocks.

After applying guard clauses, helper extraction and the Single Responsibility Principle:

- Cognitive complexity across all original functions dropped by 33% (27 → 18).
- No single function now exceeds Cognitive CC = 3, well within the 15-point threshold.
- The maximum nesting depth fell from 3 to 1.
- Four duplicate search loops were replaced by a single, testable helper.
- The code now takes less mental effort to read, review and maintain.

The system continues to manage employees Abdullah, Usman, Zayam and Bilal at Codistan with identical functionality, while being significantly more readable and maintainable..

7. Github Link

Click here (https://github.com/AbdullahNiazi7/Employee_Management_System_ComplexityAnalysis)